

11

Building a Backend with a GraphQL API

Up until now, we have only been interfacing with REST APIs. For more complex APIs that have deeply nested objects, we can use GraphQL to allow selective access to certain parts of large objects. In this chapter, we are first going to learn what GraphQL is and when it is useful. Then, we are going to experiment with making GraphQL queries and mutations. After that, we are going to implement GraphQL in a backend. Finally, we are going to briefly cover advanced GraphQL concepts.

In this chapter, we are going to cover the following main topics:

- What is GraphQL?
- Implementing a GraphQL API in a backend
- Implementing GraphQL authentication and mutations
- Overview of advanced GraphQL concepts

Technical requirements

Before we start, please install all requirements from [*Chapter 1, Preparing for Full-Stack Development*](#), and [*Chapter 2, Getting to Know Node.js and MongoDB*](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an is-

sue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch11>.

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

The CiA video for this chapter can be found at: <https://youtu.be/6gP0uM-XaVo>.

What is GraphQL?

Before we start learning how to use GraphQL, let's first focus on what GraphQL is. Like REST, it is a way to query APIs. However, it is also much more than that. GraphQL includes a server-side runtime for executing queries and a type system to define your data. It works with many database engines and can be integrated into your existing backend.

GraphQL services are created by defining types (such as a **User** type), fields on types (such as a **username** field), and functions to resolve values of fields. Let's assume we have defined the following **User** type with a function to get a username:

```
type User {  
  username: String  
}  
  
function User_username(user) {  
  return user.getUsername()  
}
```

We could then define a **Query** type and a function to get the current user:

```
type Query {  
  currentUser: User  
}  
function Query_currentUser(req) {  
  return req.auth.user  
}
```

INFO

*The **Query** type is a special type that defines the “entry point” into the GraphQL schema. It allows us to define which fields are allowed to be queried using the GraphQL API.*

Now that we have defined types with fields and functions to resolve those fields, we can make a GraphQL query to get the username of the current user. GraphQL queries look like JavaScript objects, but they only list the field names that you want to query. The GraphQL API will then return a JavaScript object that has the same structure as the query, but with the values filled in. Let's find out how a query to get the username of the current user would look like:

```
{  
  currentUser {  
    username  
  }  
}
```

That query would then return a JSON result that looks like this:

```
{  
  "data": {  
    "currentUser": {  
      "username": "dan"  
    }  
  }  
}
```

As you can see, the result has the same shape as the query. This is one of the essential concepts of GraphQL: the client can specifically ask for the fields that it needs, and the server will return exactly those fields. If we need more data about a user, we can just add new fields to the type and query.

GraphQL validates queries and results against the defined types. This ensures that we do not break the contract between the client and server. The GraphQL types serve as that contract between the client and server. After validating the query, it is executed by a GraphQL server, which then returns a result that looks exactly like the shape requested by the query. Each requested field executes a function on the server. These functions are called **resolvers**.

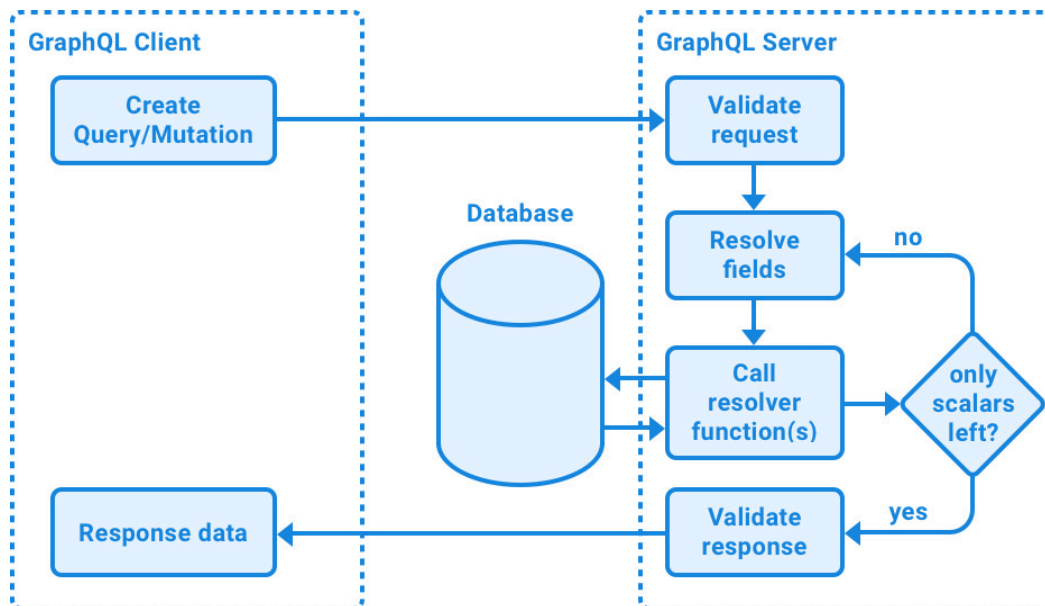


Figure 11.1 – Interaction between GraphQL client and server

Types and queries can also be deeply nested. For example, a user could have a field that returns all the posts that this user is an author of. We can then make a sub-selection of fields in those post objects as well. This works for objects within objects and even arrays of objects within objects, in multiple nesting levels. GraphQL will keep resolving fields until only simple values (scalars), such as strings and numbers are left. For example, the following query could fetch the IDs and titles of all posts that the current user created:

```
{
  currentUser {
    username
    posts {
      id
      title
    }
  }
}
```

Additionally, GraphQL allows us to define arguments for fields, which will be passed to the functions that resolve our fields. We can use arguments to, for example, get all posts with a certain tag. In GraphQL we can pass arguments to any field, even when they are deeply nested.

Arguments can even be passed to single value fields, for instance, to transform a value. For example, the following query would get a post by ID and return the post title:

```
{
  postById(id: "1234") {
    title
  }
}
```

GraphQL is especially useful if you build the backend yourself or with it in mind, as it can allow for patterns where deeply nested and interconnected data can be queried easily. However, if there are existing REST backends that you are not in control of, it is usually not worth it to add GraphQL as a separate, independent layer, due to its schema-based restrictions.

Having learned about queries, let's move on to mutations.

Mutations

In REST, any request could cause a side effect (such as writing data to the database). But, as we have learned, GET requests should only return data, and should not cause such side effects. Only POST/PUT/PATCH/DELETE re-

quests should cause data in the database to change. In GraphQL, there is a similar concept: theoretically, any field function could cause the database state to change. However, in GraphQL, we define a mutation instead of a query to explicitly state that we want to change the database state. Besides being defined with the **mutation** keyword, mutations have the same structure as queries. There is one difference, though: queries fetch fields in parallel, while mutations run in series, executing the first field function first, then the next, and so on. This behavior ensures that we do not end up with race conditions in mutations.

INFO

*In addition to the built-in **Query** type, there is also a **Mutation** type to define the allowed mutation fields.*

Now that we have learned the basics of what GraphQL is and how it works, let's get started using it in practice by implementing GraphQL in our blog application backend!

Implementing a GraphQL API in a backend

We are now going to set up GraphQL in our existing blog application backend in addition to the REST API. Doing so will allow us to see how GraphQL compares to and differs from a REST API. Follow these steps to get started setting up GraphQL on the backend:

1. Copy the existing **ch10** folder to a new **ch11** folder, as follows:

```
$ cp -R ch10 ch11
```

2. Open the **ch11** folder in VS Code.
3. First, let's install a VS Code extension to add GraphQL language support. Go to the **Extensions** tab and search for the **GraphQL.vscod-graphql** extension developed by the GraphQL Foundation. Install the extension.

- Next, install the **graphql** and **@apollo/server** libraries in the backend using the following commands:

```
$ cd backend/  
$ npm install graphql@16.8.1 @apollo/server@4.10.0
```

Apollo Server is a production-ready GraphQL server implementation that supports multiple backend web frameworks, including Express.

- Create a new **backend/src/graphql/** folder. Inside it, create a **backend/src/graphql/query.js** file, inside which we define a **Query** schema, which is the entry point of our GraphQL API (listing all supported queries for our backend), as follows:

```
export const querySchema = `#graphql  
  type Query {  
    test: String  
  }  
`
```

It is important to add a **#graphql** directive at the beginning of the template string, so that the string is recognized as GraphQL syntax and properly highlighted in a code editor. Inside our schema, we defined a **test** field, for which we define a resolver now.

- Define a **queryResolver** object which contains a function to resolve the **test** field to a static string:

```
export const queryResolver = {  
  Query: {  
    test: () => {  
      return 'Hello World from GraphQL!'  
    },  
  },  
}
```

- Create a new **backend/src/graphql/index.js** file and import the **querySchema** and **queryResolver** there:

```
import { querySchema, queryResolver } from './query.js'
```

8. Then, export an array called **typeDefs**, which includes all schemas (for now, only the query schema) and an array called **resolvers**, which contains all resolvers (for now, only the query resolver):

```
export const typeDefs = [querySchema]
export const resolvers = [queryResolver]
```

9. Edit **backend/src/app.js** and import **ApolloServer** and **expressMiddleware** from the **@apollo/server** library:

```
import { ApolloServer } from '@apollo/server'
import { expressMiddleware } from '@apollo/server/express4'
```

10. Then, import **typeDefs** and **resolvers**:

```
import { typeDefs, resolvers } from '../graphql/index.js'
```

11. After all other middleware and before the route definitions, create a new Apollo server using the schema type definitions and defined resolvers:

```
const apolloServer = new ApolloServer({
  typeDefs,
  resolvers,
})
```

12. Then, after the server is ready, mount **expressMiddleware** to a **/graphql** route, as follows:

```
apolloServer
  .start()
  .then(() => app.use('/graphql', expressMiddleware(apolloServer)))
```

13. Start the backend in development mode by running the following command:

```
$ npm run dev
```

14. Go to **http://localhost:3001/graphql** in your browser; you should see the Apollo interface to input a query on the left side, and the re-

sults on the right side.

15. Remove all comments from the editor on the left and input the following GraphQL query:

```
query ExampleQuery {  
  test  
}
```

16. Press the **Play** button to run the query, and you will see the following result:



Figure 11.2 – Successful execution of our first GraphQL query!

As you can see, our query for the `test` field returns our previously defined static string!

After implementing a basic field, let's implement some fields that access our service functions and retrieve data from MongoDB.

Implementing fields that query posts

Follow these steps to implement the fields to query posts:

1. Edit `backend/src/graphql/query.js` and import the relevant service functions:

```
import {  
  getPostById,  
  listAllPosts,  
  listPostsByAuthor,  
  listPostsByTag,  
} from '../services/posts.js'
```

2. Adjust the schema to include a **posts** field, which returns an array of posts:

```
export const querySchema = `#graphql
  type Query {
    test: String
    posts: [Post!]!
```

In GraphQL, the **[Type]** syntax means that something is an array of **Type**. We will define the **Post** type later. **Type!** is the non-null modifier and means that a type is not null (required), so **[Type!]** means that each element is a **Type**, and not **null** (the array can still be empty, though). **[Type!]!** means that the array will always exist and never be **null** (but the array can still be empty).

3. Additionally, define fields for querying posts by **author** and **tag**, both of which accept a required argument:

```
postsByAuthor(username: String!): [Post!]!
postsByTag(tag: String!): [Post!]!
```

4. Lastly, define a field to query a post by **id**:

```
postById(id: ID!): Post
}
```

5. Now that we have defined the schema, we still need to provide resolvers for all those fields. Thanks to our service functions, this is quite straightforward: we can simply call our service functions with the relevant arguments in **async** functions, as follows:

```
export const queryResolver = {
  Query: {
    test: () => {
      return 'Hello World from GraphQL!'
    },
    posts: async () => {
      return await listAllPosts()
    },
    postsByAuthor: async (parent, { username }) => {
```

```
    return await listPostsByAuthor(username)
  },
  postsByTag: async (parent, { tag }) => {
    return await listPostsByTag(tag)
  },
  postById: async (parent, { id }) => {
    return await getPostById(id)
  },
},
}
```

The resolver functions always receive the **parent** object as the first argument and an object with all arguments as a second argument.

Now we have successfully defined fields to query posts. However, the **Post** type is not defined yet, so our GraphQL queries will not work yet. Let's do that next.

Defining the Post type

After defining the **Query** type, we continue by defining the **Post** type, as follows:

1. Create a new **backend/src/graphql/post.js** file, where we import the **getUserInfoById** function to resolve the author of a post later:

```
import { getUserInfoById } from '../services/users.js'
```

2. Then, define **postSchema**. Note that **Post** consists of **id**, **title**, **author**, **contents**, **tags**, and the **createdAt** and **updatedAt** timestamps:

```
export const postSchema = `#graphql
type Post {
  id: ID!
  title: String!
  author: User
  contents: String
  tags: [String!]
  createdAt: Float
  updatedAt: Float
}
```

```
}
,
```

In this case, we use `[String!]` for the tags, and not `[String!]!`, because the `tags` field can also be non-existent/`null`.

The `createdAt` and `updatedAt` timestamps are too large to fit into a 32-bit signed integer, so their type needs to be `Float` instead of `Int`.

3. Next, define a resolver for the `author` field that gets the user using the service function:

```
export const postResolver = {
  Post: {
    author: async (post) => {
      return await getUserInfoById(post.author)
    },
  },
},
```

The resolvers for getting posts are already part of the `Query` schema, so we do not need to define how to get a post here. GraphQL knows that the query fields return `Post` arrays and then allows us to resolve further fields on the posts.

4. Edit `backend/src/graphql/index.js` and add the `postSchema` and `postResolver`:

```
import { querySchema, queryResolver } from './query.js'
import { postSchema, postResolver } from './post.js'
export const typeDefs = [querySchema, postSchema]
export const resolvers = [queryResolver, postResolver]
```

After defining the `Post` type, let's continue with the `User` type.

Defining the User type

When defining the `Post` type, we used the `User` type to define the author of a post. However, we have not defined the `User` type yet. Let's do that now:

1. Create a new **backend/src/graphql/user.js** file and import the **listPostsByAuthor** function here, as we are going to add a way to resolve the posts of a user when getting a user object, to show how GraphQL can deal with deeply nested relations:

```
import { listPostsByAuthor } from '../services/posts.js'
```

2. Define **userSchema**. Each **User** in our GraphQL schema has **username** and a **posts** field, in which we will resolve all posts that the user has written:

```
export const userSchema = `#graphql
  type User {
    username: String!
    posts: [Post!]!
  }
`
```

INFO

*We do not specify any other properties here, as we are only returning the username in our **getUserInfoById** service function. If we wanted to get the user ID here too, we would have to return it from that function. We are not just returning the full user object, as that could be a potential security vulnerability, exposing internal data such as the password (or billing info in some apps).*

3. Next, define **userResolver**, which gets all posts from the current user:

```
export const userResolver = {
  User: {
    posts: async (user) => {
      return await listPostsByAuthor(user.username)
    },
  },
}
```

4. Edit **backend/src/graphql/index.js** and add the **userSchema** and **userResolver**:

```
import { querySchema, queryResolver } from './query.js'
import { postSchema, postResolver } from './post.js'
import { userSchema, userResolver } from './user.js'
export const typeDefs = [querySchema, postSchema, userSchema]
export const resolvers = [queryResolver, postResolver, userResolver]
```

After defining the **User** type, let's try out some deeply nested queries!

Trying out deeply nested queries

Now that we have successfully defined our GraphQL schemas and resolvers, we can start querying our database using GraphQL!

For example, we can now get a full list of all posts, with their ID, title, and the username of the author, as follows:

```
query GetPostsOverview {
  posts {
    id
    title
    author {
      username
    }
  }
}
```

Execute the preceding query in the Apollo interface. As we can see, the query gets all posts, selects **id**, **title**, and **author** for each post, and then resolves **username** for each **author** instance. This query allows us to get all the data we need on the overview page in a single request, and we do not need to make separate requests to resolve the author usernames anymore!

INFO

*We did not specify the **password** field on the **User** type, so GraphQL will not allow us to access it, even if the resolver function returns a user object that*

contains the password.

Now, let's try out a query that gets a post by ID and then finds other posts by the same author. This could be used to, for example, recommend other articles to view from the same author after someone has finished reading a post:

1. We can automatically generate a query in the Apollo interface by clearing the contents of the **Operation** textbox and then selecting **Query** from **Root Types** in the **Documentation** sidebar on the left. Now click on the + button next to the **postById** field on the left, which automatically defines a query variable for us, which looks as follows:

```
query PostById($postByIdId: ID!) {  
  postById(id: $postByIdId) {
```

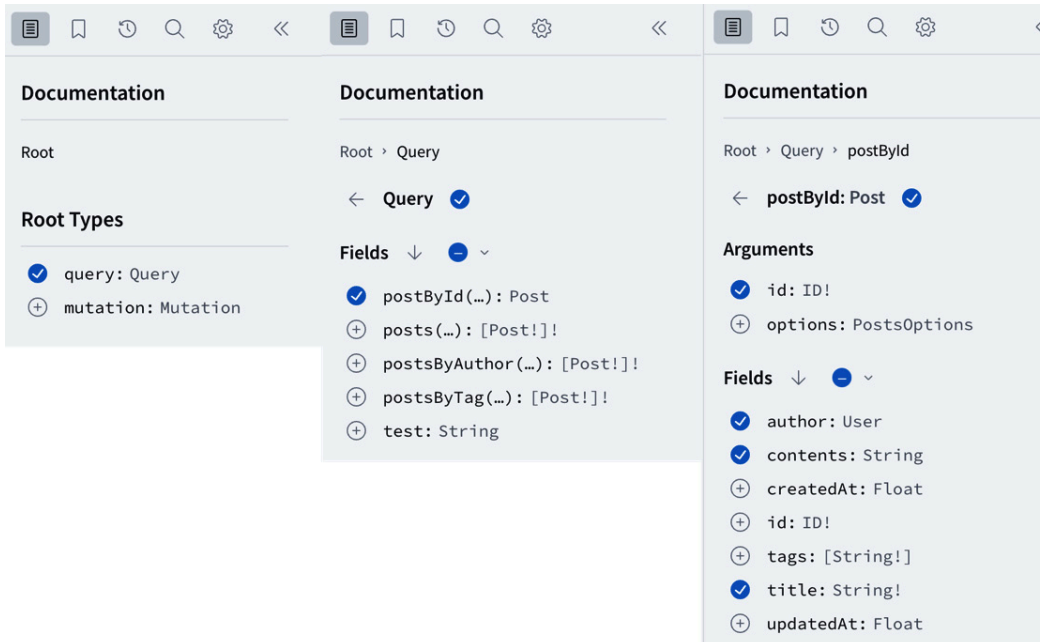


Figure 11.3 – Automatically generating a query using the Apollo interface

2. Within the post, we can now get the **title**, **contents**, and **author** values of the post:

```
title  
contents  
author {
```

3. Inside the **author** field, we get **username** and the IDs and titles of their posts:

```
    username
    posts {
      id
      title
    }
  }
}
```

4. At the bottom of the Apollo interface, there is a **Variables** section, which we need to fill with an ID that exists in our database:

```
{
  "postByIdId": "<ENTER ID FROM DATABASE>"
}
```

5. Run the query, and you will see that the post and author are resolved, and all posts written by that same author are also listed properly, as shown in the following screenshot:

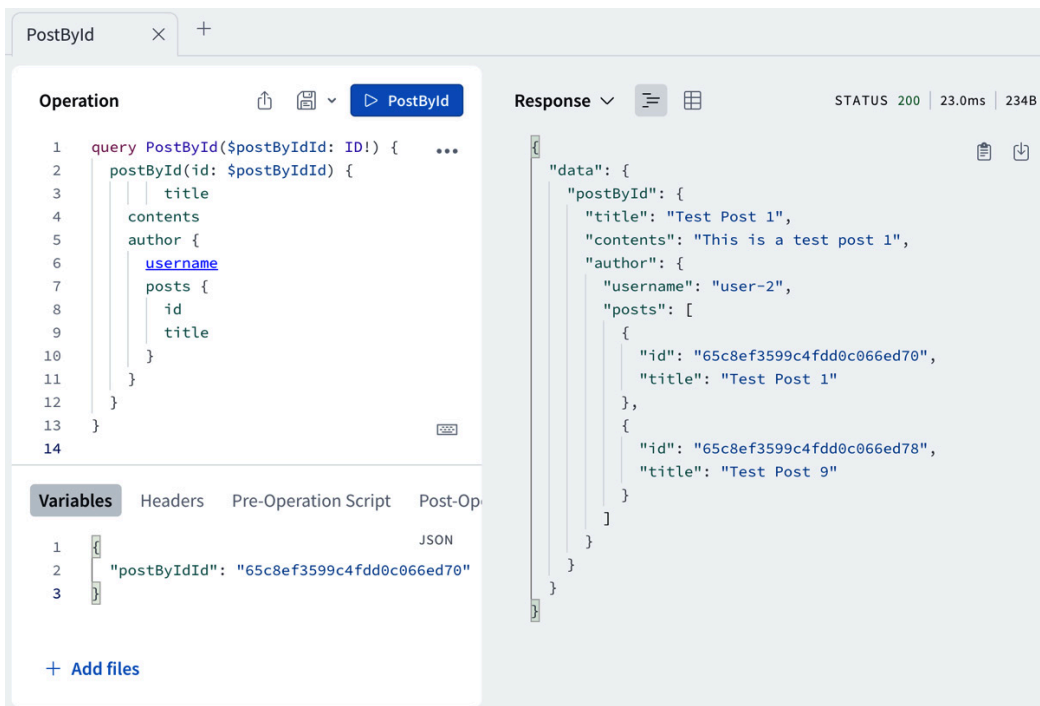


Figure 11.4 – Running deeply nested queries in GraphQL

Next, let's learn how to provide arguments to fields by defining input types.

Implementing input types

We have already learned how to define regular types in GraphQL, but what if we have a common way to provide arguments to fields? For example, the options to query posts are always the same (**sortBy** and **sortOrder**). We cannot use a regular type for this, instead, we need to define an input type. Follow these steps to implement query options in GraphQL:

1. Edit **backend/src/graphql/query.js** and define an input type in the schema:

```
export const querySchema = `#graphql
  input PostsOptions {
    sortBy: String
    sortOrder: String
  }
```

2. Then, use the input type as an argument to fields, as follows:

```
type Query {
  test: String
  posts(options: PostsOptions): [Post!]!
  postsByAuthor(username: String!, options: PostsOptions): [Post!]!
  postsByTag(tag: String!, options: PostsOptions): [Post!]!
  postById(id: ID!, options: PostsOptions): Post
}
```

3. Now, edit the resolvers to pass on **options** to the service functions:

```
posts: async (parent, { options }) => {
  return await listAllPosts(options)
},
postsByAuthor: async (parent, { username, options }) => {
  return await listPostsByAuthor(username, options)
},
postsByTag: async (parent, { tag, options }) => {
```

```
    return await listPostsByTag(tag, options)
  },
```

4. Try out the following query to see whether the posts are sorted properly:

```
query SortedPosts($options: PostsOptions) {
  posts(options: $options) {
    id
    title
    createdAt
    updatedAt
  }
}
```

5. Set the following variables:

```
{
  "options": {
    "sortBy": "updatedAt",
    "sortOrder": "ascending"
  }
}
```

6. Run the query by pressing the **Play** button, and you should see that the response is sorted by the **updatedAt** timestamp ascending!

Now that we have successfully implemented functionality to query our database using GraphQL, let's move on to implementing a way to create a new post using GraphQL Mutations.

Implementing GraphQL authentication and mutations

We are now going to implement a way to create new posts using GraphQL. To define fields that change the database state, we need to create them under the **mutation** type. Before we can do that, however, we first need to implement authentication in GraphQL, so that we can access the currently logged-in user when creating a post.

Adding authentication to GraphQL

Because we are using GraphQL with Express, we can use any Express middleware with GraphQL and pass it to our resolvers as **context**. As such, we can use the existing **express-jwt** middleware to parse the JWT. Let's get started adding authentication to GraphQL now:

1. Our current configuration of the **requireAuth** middleware ensures that the user is logged in and throws an error if they are not. However, this is an issue when passing the **auth** context to GraphQL, because not all queries require authentication. We are now going to create a new **optionalAuth** middleware that does not require credentials to process a request. Edit **backend/src/middleware/jwt.js** and define the following new middleware:

```
export const optionalAuth = expressjwt({
  secret: () => process.env.JWT_SECRET,
  algorithms: ['HS256'],
  credentialsRequired: false,
})
```

2. Now, edit **backend/src/app.js** and import the **optionalAuth** middleware there:

```
import { optionalAuth } from './middleware/jwt.js'
```

3. Edit the **app.use()** call where we defined the **/graphql** route and add the **optionalAuth** middleware to it, similarly to how we did it for routes:

```
apolloServer.start().then(() =>
  app.use(
    '/graphql',
    optionalAuth,
```

4. Then, add a second argument to the Apollo **expressMiddleware**, defining a **context** function that provides **req.auth** to the GraphQL resolvers as context:

```

    expressMiddleware(apolloServer, {
      context: async ({ req }) => {
        return { auth: req.auth }
      },
    }),
  ),
)

```

Next, let's move on to implementing mutations in GraphQL.

Implementing mutations

Now that we have added authentication to GraphQL, we can define our mutations. Follow these steps to create mutations for signup, login, and creating posts:

1. Create a new **backend/src/graphql/mutation.js** file and import **GraphQLError** (for throwing an **UNAUTHORIZED** error when the user is not logged in), as well as the **createUser**, **loginUser**, and **createPost** functions:

```

import { GraphQLError } from 'graphql'
import { createUser, loginUser } from '../services/users.js'
import { createPost } from '../services/posts.js'

```

2. Define **mutationSchema**, in which we first define fields to sign up and log in users. The **signupUser** field returns a user object, and the **loginUser** field returns a JWT:

```

export const mutationSchema = `#graphql
type Mutation {
  signupUser(username: String!, password: String!): User
  loginUser(username: String!, password: String!): String

```

3. Then, define a field to create a new post from some given **title**, **contents** (optional), and **tags** (optional). It returns a newly created post:

```

  createPost(title: String!, contents: String, tags: [String]): Post

```

```

    }
  },

```

4. Define the resolver, in which we first define the **signupUser** and **loginUser** fields, which are quite straightforward:

```

export const mutationResolver = {
  Mutation: {
    signupUser: async (parent, { username, password }) => {
      return await createUser({ username, password })
    },
    loginUser: async (parent, { username, password }) => {
      return await loginUser({ username, password })
    },
  },

```

5. Next, we define the **createPost** field. Here, we first access the arguments passed to the field, and as a third argument to the resolver function, we get the context we created earlier:

```

    createPost: async (parent, { title, contents, tags }, { auth }) => {

```

6. If the user is not logged in, the **auth** context will be **null**. We throw an error in that case and do not create a new post:

```

    if (!auth) {
      throw new GraphQLError(
        'You need to be authenticated to perform this action.',
        {
          extensions: {
            code: 'UNAUTHORIZED',
          },
        },
      )
    }

```

7. Otherwise, we use **auth.sub** (which contains the user ID) and the provided arguments to create a new post:

```

    return await createPost(auth.sub, { title, contents, tags })
  },

```

```
  },
}
```

8. Edit **backend/src/graphql/index.js** and add the **mutationSchema** and **mutationResolver**:

```
import { querySchema, queryResolver } from './query.js'
import { postSchema, postResolver } from './post.js'
import { userSchema, userResolver } from './user.js'
import { mutationSchema, mutationResolver } from './mutation.js'
export const typeDefs = [querySchema, postSchema, userSchema, mutationSchema]
export const resolvers = [
  queryResolver,
  postResolver,
  userResolver,
  mutationResolver,
]
```

After implementing mutations, let's learn how to use them.

Using mutations

After defining the possible mutations, we can use them by running them in the Apollo interface. Follow these steps to first sign up a user, then log them in, and finally create a post – all using GraphQL:

1. Go to **<http://localhost:3001/graphql>** to view the Apollo interface.

Define a new mutation that signs up a user with a given username and password, and returns the username if the signup was successful:

```
mutation SignupUser($username: String!, $password: String!) {
  signupUser(username: $username, password: $password) {
    username
  }
}
```

TIP

You can use the **Documentation** section on the left by going back to **Root Types**, clicking on **Mutation**, and then clicking on the + icon next to **signupUser**. Then, click on the + icon next to the **username** field. This will automatically create the preceding code.

2. Edit the variables at the bottom and enter a username and password:

```
{
  "username": "graphql",
  "password": "gql"
}
```

3. Execute the **SignupUser** mutation by pressing the play button.
4. Next, create a new mutation to log in a user:

```
mutation LoginUser($username: String!, $password: String!) {
  loginUser(username: $username, password: $password)
}
```

5. Enter the same variables as before and press the play button, the response contains a JWT. Copy and store the JWT somewhere for later use.
6. Define a new mutation to create a post. This mutation returns **Post**, so we can get the **id**, **title**, and **username** values for **author**:

```
mutation CreatePost($title: String!, $contents: String, $tags: [String]) {
  createPost(title: $title, contents: $contents, tags: $tags) {
    id
    title
    author {
      username
    }
  }
}
```

This is an example of where GraphQL really shines. We can resolve the username of the author after creating the post to see whether it was really created with the correct user, because we can access the resolvers defined for **Post**, even in mutations! As you can see, GraphQL is very flexible.

7. Enter the following variables:

```
{
  "title": "GraphQL Post",
  "contents": "This is posted from GraphQL!"
}
```

8. Select the **Headers** tab, press the **New header** button, enter **Authorization** for header key, and **Bearer <Paste previously copied JWT here>** for value. Then press the **Play** button to submit the mutation.

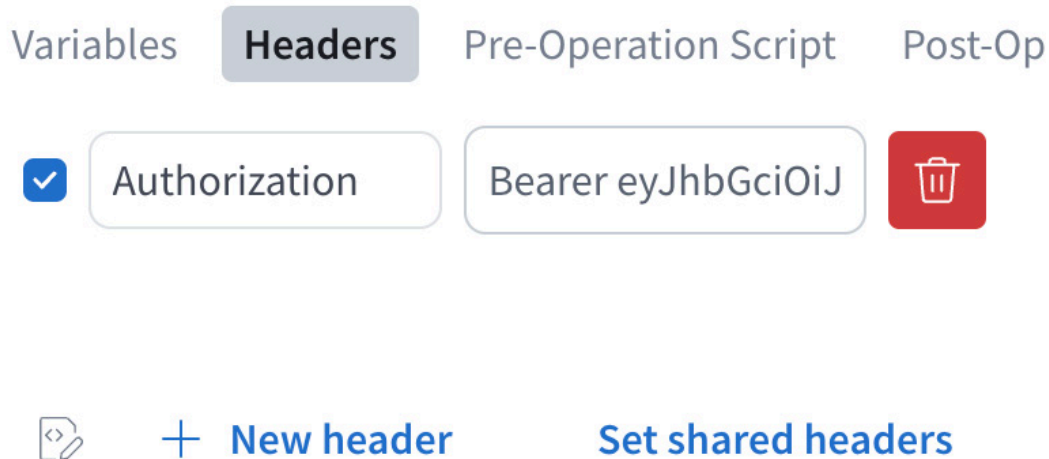


Figure 11.5 – Adding the Authorization header in the Apollo interface

9. In the response, you can see that the post was successfully created, and the author is set and resolved correctly!

Having implemented GraphQL queries and mutations for our blog applications, let's wrap up the chapter by giving an overview of advanced GraphQL concepts.

Overview of advanced GraphQL concepts

Out of the box, GraphQL comes with a set of scalar types:

- **Int**: A signed 32-bit integer
- **Float**: A signed double-precision floating-point value

- **String:** A UTF-8 encoded character sequence
- **Boolean:** Can be true or false
- **ID:** A unique identifier, serialized as a **String**, but meant to signify that it is not human readable

GraphQL also allows the definition of enums, which are a special kind of scalar. They are restricted to certain values. For example, we could have the following enum to distinguish different types of posts:

```
enum PostType {  
  UNPUBLISHED,  
  UNLISTED,  
  PUBLIC  
}
```

In Apollo, enums will be handled as strings that can only have certain values, but this may be different in other GraphQL implementations.

Many GraphQL implementations also allow defining custom scalar types. Apollo, for example, supports the definition of custom scalar types.

Fragments

When the same kind of fields are regularly accessed, we can create a fragment to simplify and standardize access to them. For example, if we often resolve users, and users have fields such as **username**, **profilePicture**, **fullName**, and **biography**, we could create the following fragment:

```
fragment UserInfo on User {  
  username  
  profilePicture  
  fullName  
  biography  
}
```

This fragment can then be used in queries. For example, see this snippet:

```
{
  posts {
    author {
      ...UserInfo
    }
  }
}
```

Fragments are especially useful when the same kind of field structure is used multiple times in the same query. For example, if an author had **followedBy** and **follows** fields, we could resolve all users like this:

```
{
  posts {
    author {
      ...UserInfo
      followedBy {
        ...UserInfo
      }
      follows {
        ...UserInfo
      }
    }
  }
}
```

Introspection

Introspection allows us to query the defined schemas themselves to get a feeling for the data that the server can provide for us. It is essentially querying the schemas defined by the GraphQL server. We can use the **__schema** field to get all schemas. A schema consists of **types**, which have **name** values.

For example, we can use the following query to get all types defined by our server:

```
{
```

```
__schema {  
  types {  
    name  
  }  
}
```

If you execute this query on our server, you will get (among other types) our defined **Query**, **Post**, **User**, and **Mutation** types.

Introspection queries are very powerful, and you can get a lot of information about the possible queries and mutations from it. Actually, the Apollo interface uses introspection to render the **Documentation** sidebar and to auto-complete fields for us!

Summary

In this chapter, we learned what GraphQL is and how it can be more flexible than REST while requiring less boilerplate code, especially when querying deeply nested objects. Then, we implemented GraphQL in our backend and created various types, queries, and mutations. We also learned how to integrate JWT authentication in GraphQL. Finally, we wrapped up the chapter by learning about advanced concepts, such as the type system, fragments, and introspection.

In the next chapter, [***Chapter 12**, Interfacing with GraphQL on the Frontend Using Apollo Client*](#), we are going to learn how to access and integrate GraphQL in our frontend using React and the Apollo Client library.